HW 4

Part 1: Linear Regressions

Perform the following activities in Python:

1. Get the dataset - pick any one of the simple regressions dataset from the Dataset folder.
2. Load the .csv file into a data frame in Python.
3. Describe the data, provide information on the dataset.
4. Conduct Linear Regression.
5. Preprocess data if needed.
6. Print the correlation coefficient matrix.
7. Generate heatmap.
8. Normalize data using the min-max scaling.
9. Train the model - Randomly sample 80% of each dataset and save them in a training dataset and save the remaining 20% of each dataset in a testing dataset.
10. Split the list of labels accordingly.
11. Make Predictions.
12. Evaluate the model.
13. Generate relevant graphs.

In [1]:
```
import pandas as pd
import numpy as np
import seaborn as sns
import matplotlib.pyplot as plt
from sklearn.linear_model import LinearRegression
```

1. Get the dataset - pick any one of the simple regressions dataset from the Dataset folder.

I chose the "Salary_dataset.csv"

2. Load the .csv file into a data frame in Python.

In [16]:
```
# Load dataset
df = pd.read_csv("Salary_dataset.csv")

# Display the first 5 rows
df.head()
```

Out[16]:

	Unnamed: 0	YearsExperience	Salary
0	0	1.2	39344.0
1	1	1.4	46205.0
2	2	1.6	37732.0
3	3	2.1	43526.0
4	4	2.3	39892.0

3. Describe the data, provide information on the dataset.

In [30]:
```
# Dataset information
# Show structure of dataset (columns, data types, non-null values)
print(df.info())

# Show statistical summary
print(df.describe())

# Check for missing values
print(df.isnull().sum())
```

```
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 30 entries, 0 to 29
Data columns (total 4 columns):
 #   Column             Non-Null Count  Dtype
---  ------             --------------  -----
 0   Unnamed: 0         30 non-null     int64
 1   YearsExperience    30 non-null     float64
 2   Salary             30 non-null     float64
 3   YearsExperience_scaled  30 non-null     float64
dtypes: float64(3), int64(1)
memory usage: 1.1 KB
None
       Unnamed:  0  YearsExperience        Salary  YearsExperience_scaled
count    30.000000        30.000000     30.000000               30.000000
mean     14.500000         5.413333  76004.000000                0.446227
std       8.803408         2.837888  27414.429785                0.301903
min       0.000000         1.200000  37732.000000                0.000000
25%       7.250000         3.300000  56731.750000                0.221404
50%      14.500000         4.800000  65238.000000                0.382979
75%      21.750000         7.800000  100545.750000               0.702128
max      29.000000        10.600000  122392.000000               1.000000
Unnamed: 0                 0
YearsExperience            0
Salary                     0
YearsExperience_scaled     0
dtype: int64
```

Dataset Information:

There are 30 observations or row and 2 columns. The 2 columns are YearsExperience, the independent variable, and Salary, the dependent variable. There are no missing values.

4. Conduct Linear Regression.

In [5]:
```
from sklearn.linear_model import LinearRegression
```

5. Preprocess data if needed.

No preprocessing is required because there are no missing values or categorical variables.

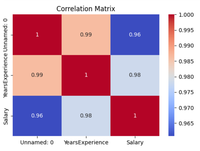
6. Print the correlation coefficient matrix.

In [18]:
```
# Compute correlation between variables
corr_matrix = df.corr()

# Print correlation matrix
print(corr_matrix)
```

```
                 Unnamed:  0  YearsExperience    Salary
Unnamed: 0          1.000000         0.986460  0.960826
YearsExperience     0.986460         1.000000  0.978242
Salary              0.960826         0.978242  1.000000
```

7. Generate heatmap.

In [20]:
```
# Create a figure with specific size
plt.figure(figsize=(6,4))

# Generate heatmap with correlation values displayed
sns.heatmap(corr_matrix, annot=True, cmap="coolwarm")

# Add title to heatmap
plt.title("Correlation Matrix")

# Display the heatmap
plt.show()
```



8. Normalize data using the min-max scaling.

In [22]:
```
# Import MinMaxScaler for normalization
from sklearn.preprocessing import MinMaxScaler

# Create scaler object
scaler = MinMaxScaler()

# Scale YearsExperience column between 0 and 1
df['YearsExperience_scaled'] = scaler.fit_transform(df[['YearsExperience']])
```

9. Train the model - Randomly sample 80% of each dataset and save them in a training dataset and save the remaining 20% of each dataset in a testing dataset.

In [26]:
```
# Import function to split dataset
from sklearn.model_selection import train_test_split

# Define independent variable (feature)
X = df[['YearsExperience_scaled']]

# Define dependent variable (target)
y = df['Salary']

# Split data into training (80%) and testing (20%)
X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.2, random_state=42
)
```

10. Split the list of labels accordingly.

I already split the list of label accordingly above.

11. Make Predictions.

In [11]:
```
# Import linear Regression model
from sklearn.linear_model import LinearRegression

# Create model
# Create linear Regression object
model = LinearRegression()

# Train model using training data
model.fit(X_train, y_train)

# Predict salaries for test dataset
y_pred = model.predict(X_test)
```

12. Evaluate the model.

In [27]:
```
# Import evaluation metrics
from sklearn.metrics import mean_squared_error, r2_score

# Calculate Mean Squared Error
mse = mean_squared_error(y_test, y_pred)

# Calculate Root Mean Squared Error
rmse = np.sqrt(mse)

# Calculate R-squared value
r2 = r2_score(y_test, y_pred)

# Print evaluation results
print("MSE:", mse)
print("RMSE:", rmse)
print("R2 Score:", r2)
```

```
MSE: 49830096.85590839
RMSE: 7059.0436219051055
R2 Score: 0.9024661774180499
```

Our model predicts 90% if variance correctly which means that the model is performing well overall.

13. Generate relevant graphs.

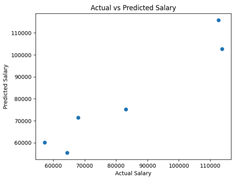
1. Regression Line (Using Original Values)

In [29]:
```
# Plot scatter plot of actual data points using original values
plt.scatter(df['YearsExperience'], df['Salary'], color='blue')

# Plot regression line/best fit line
plt.plot(
    df['YearsExperience'],
    model.predict(df[['YearsExperience_scaled']]),
    color='red'
)

# Label x-axis
plt.xlabel("Years of Experience")

# Label y-axis
plt.ylabel("Salary")

# Add title
plt.title("Salary vs Experience (Linear Regression)")

# Show plot
plt.show()
```

The linear regression graph shows a strong positive relationship. It shows that as years of experience increase, salary increases proportionally.

2. Actual vs Predicted

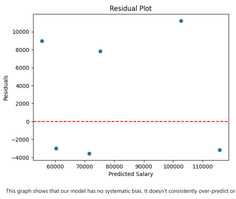
In [14]:
```
plt.scatter(y_test, y_pred)
plt.xlabel("Actual Salary")
plt.ylabel("Predicted Salary")
plt.title("Actual vs Predicted Salary")
plt.show()
```



The points are close to where a diagonal line would be if we drew it from the bottom left to the top right of the graph. Our model is predicting accurately across different salary ranges. This shows that our model performs well on unseen data which validates the strong R^2 score of 0.90.

3. Residual Plot

In [15]:
```
# Calculate residuals (difference between actual and predicted)
residuals = y_test - y_pred

# Scatter plot comparing actual vs predicted salaries
plt.scatter(y_pred, residuals)

# Draw horizontal line at zero error
plt.axhline(y=0, color='red', linestyle='--')

# Label axes
plt.xlabel("Predicted Salary")
plt.ylabel("Residuals")

# Add title
plt.title("Residual Plot")

# Display plot
plt.show()
```



This graph shows that our model has no systematic bias. It doesn't consistently over-predict or under-predict. The random scattering of points with small errors indicates a well-fitted linear model and validates the performance metrics.